

VLLM专题（三十八）—vLLM 的 torch.compile 集成



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

在vLLM的V1架构中，`torch.compile` 默认启用，并且是框架的关键部分。本文档通过一个简单的示例来展示如何理解 `torch.compile` 的使用。

在整个示例中，我们将使用v1运行一个常见的Llama模型，并开启调试级别的日志记录以显示所有细节。使用的命令是：

```
VLLM_USE_V1=1 VLLM_LOGGING_LEVEL=DEBUG vllm serve meta-llama/Llama-3.2-1B
```

1. 编译缓存

在非常详细的日志中，我们可以看到：

```
INFO 03-07 03:06:55 [backends.py:409] Using cache directory: ~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0 for vLLM's torch.compile
```

vLLM会考虑所有相关因素，并决定一个目录来存储所有的编译产物。这意味着，您可以直接在部署场景中复制整个 `~/.cache/vllm/torch_compile_cache` 目录，以节省大量的编译时间，从而加速vLLM实例的启动时间。

考虑的因素包括：

- 所有相关的配置（参见 `config.py` 中的 `compute_hash` 函数）
- PyTorch配置（参见 `compiler_interface.py` 中的 `compute_hash` 函数）
- 模型的 `forward` 函数以及 `forward` 函数调用的相关函数（见下文）

考虑到所有这些因素，通常我们可以保证缓存是安全可用的，并且不会导致任何意外行为。因此，缓存默认是启用的。如果您想调试编译过程，或者怀疑缓存导致了一些问题，可以通过设置环境变量 `VLLM_DISABLE_COMPILE_CACHE=1` 来禁用缓存。

vLLM的 `torch.compile` 集成的一个独特之处在于，我们保证所有编译在服务任何请求之前完成。不会有任何请求触发新的编译。否则，引擎将会被该请求阻塞，响应时间会出现意外的峰值。

2. Python代码编译

在非常详细的日志中，我们可以看到以下内容：

```
DEBUG 03-07 03:06:52 [decorators.py:203] Start compiling function <code object forward at
0x7f08acf40c90, file "xxx/vllm/model_executor/models/llama.py", line 339>

DEBUG 03-07 03:06:54 [backends.py:370] Traced files (to be considered for compilation cache):

DEBUG 03-07 03:06:54 [backends.py:370] xxx/torch/_dynamo/polyfills/builtins.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/torch/nn/modules/container.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/torch/nn/modules/module.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/attention/layer.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/distributed/communication_op.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/distributed/parallel_state.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/custom_op.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/layers/activation.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/layers/layernorm.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/layers/linear.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/layers/rotary_embedding.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/layers/vocab_parallel_embedding.py
DEBUG 03-07 03:06:54 [backends.py:370] xxx/vllm/model_executor/models/llama.py

DEBUG 03-07 03:07:07 [backends.py:462] Computation graph saved to ~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/computation_graph.py
DEBUG 03-07 03:07:07 [wrapper.py:105] Dynamo transformed code saved to ~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/transformed_code.py
```

这部分内容涉及Python代码的编译，即通过Dynamo进行图捕获。它尝试对代码 `xxx/vllm/model_executor/models/llama.py:339` 中的函数进行追踪，这是我们编译的模型的 `forward` 函数。在前向传播过程中，Dynamo还会调用和内联其他函数，如日志所示，包括来自 `xxx/torch/nn/modules/module.py` 的一些PyTorch函数（由PyTorch的 `nn.Module` 使用，因为模块属性访问会触发函数调用），以及来自vLLM的一些通信/注意力/激活函数。在决定使用哪个缓存目录时，所有被追踪的文件都会被考虑。这样一来，上述文件中的任何代码更改都会触发编译缓存未命中，从而导致重新编译。

Dynamo编译的结果是一个新函数，存储

在 `~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/transformed_code.py` 中。通常，这个函数会从模块中解包张量，然后将其传递给追踪的计算图。计算图存储

在 `~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/computation_graph.py` 中。

3. 计算图处理

计算图中为每个张量都标注了形状。输入包括输入ID、位置ID、模型的权重和缓冲区，输出是最终的隐藏状态。需要注意的是，语言模型头部的投影和采样操作不在计算图的考虑范围内。

计算图的大多数输入具有静态形状，因为它们是模型的权重和缓冲区，在模型的整个生命周期中不会改变。只有输入ID和位置ID具有符号形状，即它们的形状可能会因批次不同而变化。然而，它们会共享相同的符号形状。也就是说，计算图中唯一变化的大小是批次大小（当前前向传播中处理的token数量）。

注意力操作较为复杂，它需要与键值缓存（kv caches）交互，且形状复杂。幸运的是，注意力操作的输出与注意力操作的输入查询共享相同的形状。因此，我们将整个注意力操作封装到一个PyTorch自定义操作 `torch.ops.vllm.unified_attention_with_output` 中，这样Dynamo就不会尝试检查任何内部操作。通过这种方式，尽管注意力操作很复杂，但从Dynamo的角度来看，我们仍然可以捕获模型的计算图作为一个完整的图。

计算图会进一步被 `split_ops`（通常是注意力操作）分割成多个部分。因此，在 `~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/computation_graph.py` 文件中，我们可以看到许多子模块，每个子模块都是分割后的一部分图：

- 注意力操作本身是一个子模块。
- 从一个注意力操作到下一个注意力操作之间的计算图部分是一个子模块。

每个子模块可以通过其索引进行标识，并将被单独处理。

4. 计算图编译

在非常详细的日志中，我们还可以看到以下内容：

```
DEBUG 03-07 03:52:37 [backends.py:134] store the 0-th graph for shape None from inductor
via handle ('fpegyiq3v3wzjzphd45wkflpabggdbjpylgr7tta4hj6uplstsiv', '~/.cache/vllm/torch_
compile_cache/1517964802/rank_0_0/inductor_cache/iw/ciwzrk3ittdqatuzwonnajywvno3llvjcs2vf
dldzwzozn3zi3iy.py')
DEBUG 03-07 03:52:39 [backends.py:134] store the 1-th graph for shape None from inductor
via handle ('f7fmlodmf3h3by5iiu2c4zarwoxbg4eytwr3ujdd2jphl4pospfd', '~/.cache/vllm/torch_
compile_cache/1517964802/rank_0_0/inductor_cache/ly/clyfzxldfsj7ehaluis2mca2omqka4r7mgced
lf6xfjh645nw6k2.py')
...
DEBUG 03-07 03:52:45 [backends.py:134] store the 15-th graph for shape None from inductor
via handle ('f7fmlodmf3h3by5iiu2c4zarwoxbg4eytwr3ujdd2jphl4pospfd', '~/.cache/vllm/torch_
compile_cache/1517964802/rank_0_0/inductor_cache/ly/clyfzxldfsj7ehaluis2mca2omqka4r7mgce
dlf6xfjh645nw6k2.py')
DEBUG 03-07 03:52:45 [backends.py:134] store the 16-th graph for shape None from inductor
via handle ('fvj3ccoi7m34f3dnr4itmu55mmun44l5xymwhrjllwisylsk7q6jy', '~/.cache/vllm/torch_
compile_cache/1517964802/rank_0_0/inductor_cache/tf/ctffftkg1j7b4lcttq5cymx6cew372uoauupq
n6ldsvpiucavqcjc.py')
```

这意味着第一段计算图（形状为 `None`，表示符号形状）由Inductor编译（使用键 `fpegylq3v3wzjzphd45wkflpabggdbjpylgr7tta4hj6uplstsiv`）。编译后的内核存储在 `~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/inductor_cache/iw/ciwzrk3ittdqatuzwonnajywvno3llvjcs2vfdldzwzozn3zi3iy.py` 中。您可以打开该文件查看Inductor最终运行的代码。

另一个细节是：您可以看到第1段图和第15段图具有相同的键，而第0段图和第16段图则不同。这是预期的，因为我们通过注意力操作将图分割，得到了3个独特的子图：

1. 注意力操作之前的第一层
2. 每个中间层，从一个注意力操作到下一个注意力操作
3. 注意力操作之后的最后一层

如果我们已经拥有缓存目录（例如第二次运行相同的代码），我们将看到以下日志：

```
DEBUG 03-07 04:00:45 [backends.py:86] Directly load the 0-th graph for shape None from inductor via handle ('fpegylq3v3wzjzphd45wkflpabggdbjpylgr7tta4hj6uplstsiv', '~/.cache/vllm/torch_compile_cache/1517964802/rank_0_0/inductor_cache/iw/ciwzrk3ittdqatuzwonnajywvno3llvjcs2vfdldzwzozn3zi3iy.py')
```

这一次，Inductor编译被完全跳过，我们将从磁盘加载上次获得的编译产物。

上述示例仅使用Inductor编译通用形状（即符号形状）。我们还可以使用Inductor编译某些特定形状，例如：

```
VLLM_USE_V1=1 vllm serve meta-llama/Llama-3.2-1B --compilation_config '{"compile_sizes': [1, 2, 4, 8]}"
```

然后，它还会为批次大小为1、2、4、8编译特定的内核。此时，计算图中的所有形状都是静态且已知的，我们将启用自动调优以追求最大性能。第一次运行时可能会比较慢，但下次运行时，我们可以直接跳过调优并运行已调优的内核。

当所有形状都已知时，`torch.compile`可以比较不同的配置，并通常会找到一些更好的配置来运行内核。例如，我们可以看到以下日志：

```
AUTOTUNE mm(8x2048, 2048x3072)
  triton_mm_4 0.0130 ms 100.0% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=128, BLOCK_M=16, BLOCK_N=32, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=5, num_warps=2
  triton_mm_8 0.0134 ms 97.4% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=128, BLOCK_M=16, BLOCK_N=64, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=5, num_warps=4
  triton_mm_12 0.0148 ms 87.7% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=128, BLOCK_M=16, BLOCK_N=128, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=4, num_warps=4
  mm 0.0160 ms 81.6%
  triton_mm_16 0.0165 ms 78.7% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=64, BLOCK_M=16, BLOCK_N=128, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=5, num_warps=8
  triton_mm_3 0.0199 ms 65.4% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=32, BLOCK_M=16, BLOCK_N=32, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=5, num_warps=2
  triton_mm_1 0.0203 ms 64.2% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=128, BLOCK_M=16, BLOCK_N=32, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=2, num_warps=2
  triton_mm_7 0.0203 ms 64.1% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=64, BLOCK_M=16, BLOCK_N=64, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=3, num_warps=4
  triton_mm_2 0.0208 ms 62.5% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=32, BLOCK_M=16, BLOCK_N=64, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=5, num_warps=4
  triton_mm_11 0.0215 ms 60.5% ACC_TYPE='t1.float32', ALLOW_TF32=False, BLOCK_K=64, BLOCK_M=16, BLOCK_N=128, B_PROLOGUE_CAST_TYPE=None, EVEN_K=True, GROUP_M=8, num_stages=3, num_warps=4
SingleProcess AUTOTUNE benchmarking takes 2.0428 seconds and 7.5727 seconds precompiling
```


这意味着，对于形状为8x2048x3072的矩阵乘法，`torch.compile` 尝试了多种配置的Triton模板，并且它比默认代码（调用cublas库）快得多。

不幸的是，由于自动调优需要相当长的时间（从几秒到几分钟，取决于模型大小和批次大小），尽管它可以缓存以供以后使用，但为了用户友好性，我们默认关闭了它。如果您希望获得最大性能，建议通过编译特定形状来尝试启用它。

5. Cudagraph 捕获

vLLM 的 V1 架构使用了分段 cudagraph。完整的计算图如上所述被拆分，我们仅捕获位于注意力操作之间的图部分（包括任何注意力操作之前的第一个图和所有注意力操作之后的最后一个图）。这是基于一个常见的观察：注意力操作之间的计算通常是逐 token 的，并且容易通过 cudagraph 处理；而注意力操作本身较为复杂，不容易与 cudagraph 兼容。因此，通过在 eager 模式下运行注意力操作，同时将其余操作放入 cudagraph 中，我们保持了注意力操作的灵活性。

分段 cudagraph 还具有精细的内存管理。其目的是仅排除注意力内核的部分，而将所有其他模块和内存分配操作保持在 cudagraph 中。这就是为什么在 V1 中，注意力操作的输出 tensor 作为输入传递给注意力操作的原因。

cudagraph 的捕获和管理由编译器后端完成，并且当批量大小有对应的 cudagraph 被捕获时，会进行重放。模型的调用者（模型运行者）只需要确保正确管理输入缓冲区。所有中间缓冲区会由编译器后端自动管理。

默认情况下，vLLM 会尝试确定一组大小来捕获 cudagraph。你也可以通过配置

`cudagraph_capture_sizes` 来覆盖这个设置：

```
VLLM_USE_V1=1 vllm serve meta-llama/Llama-3.2-1B --compilation_config '{"cudagraph_capture_sizes': [1, 2, 4, 8]}"
```

这时，它只会捕获指定大小的 cudagraph。这样可以更精细地控制 cudagraph 的捕获过程。